OBJECT ORIENTED PROGRAMMING
CCS20704, DCS20504, CCS23803

Received Date : 3 September 2026
Submission Due Date: 23 October 2026
: Dr. Jamal Abdullahi
Weightage : 15%
Semester : SEPTEMBER 2026

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:	
I declare that: • ***This assignment is my/our own work.*** • ***I/we understand what is meant by plagiarism.*** • ***My lecturer has the right to deduct my marks in case of:*** • ***Late submission*** • ***Any plagiarism found in my assignment.***	
NAME: SAFANA MARIYAM BINTI HAJA MAIDEEN **MATRICS NO: 012026070737** 	**NAME: DHIYA ZULAIKHA BINTI ZULHAILMY** **MATRICS NO: 012026070735** 

1.0 INTRODUCTION

In this assignment, we were asked to design classes for objects of our choice using Object-Oriented Programming (OOP) in Java. We chose a café setting and built a simple ordering system for a coffee shop called Kopi Senja. The idea behind OOP is to model real-world things as objects, where each object keeps its own data together with the actions that can be performed on that data. In a café, the two most important objects are the drink being sold and the customer buying it, so we created a Drink class and a Customer class.

A key part of this assignment is encapsulation. Instead of letting other parts of the program change an object's data directly, we made every attribute private, so access only happens through public getter and setter methods. This allows the setters to check a value before accepting it, so a negative price, quantity or age cannot enter the system. Each class has seven private attributes, a default constructor, a parameterized constructor, and a getter and setter for every attribute. We also added methods that perform useful work with the data. The Drink class calculates the subtotal of an order and checks whether a drink is available, while the Customer class calculates the member discount and adds loyalty points after a purchase.

Besides the two classes, we wrote a main class called KopiSenjaMain, where the user interacts with the objects. The program asks for the customer's name and membership status, then displays a menu so the customer can order drinks until they choose to check out. To control this flow, we used if/else and switch statements for decision making, and a do-while loop to repeat the menu. Through this assignment, we gained a clearer understanding of how classes, objects and encapsulation work together in a real program.

1.1 Project Objectives

- To design two Java classes that model real café objects, each with at least seven private attributes.
- To apply encapsulation by controlling access to data through getter and setter methods.
- To implement default and parameterized constructors, and additional methods that perform meaningful operations.
- To build an interactive main class that uses selection and looping structures.
- To test drink availability, order calculations, membership discounts and loyalty points.

1.2 Overview of Kopi Senja

The program offers the following menu. Members receive a 10% discount on their order and earn one loyalty point for every full RM10 spent after the discount. Non-members receive no discount or points.

Option	Drink	Price	Status
1	Iced Latte	RM12.00	Available
2	Cappuccino	RM10.00	Available
3	Americano	RM8.00	Unavailable (sold out)
4	Checkout	–	Ends the order and prints the receipt

1.3 Project Scope

Kopi Senja is a console application that serves one customer per session. Online payment, inventory tracking and permanent data storage are outside the scope of this assignment, as the aim is to demonstrate the required OOP concepts rather than to build a complete point-of-sale system.

2.0 CLASS DESIGN AND ATTRIBUTES

We identified two real-world objects that every café order involves: the **drink** being sold and the **customer** buying it. Drink models an item on the menu and knows its own price and availability. Customer models the buyer and knows whether they are a member. A third class, KopiSenjaMain, acts as the controller that creates these objects and lets the user interact with them, as shown in Figure 1.

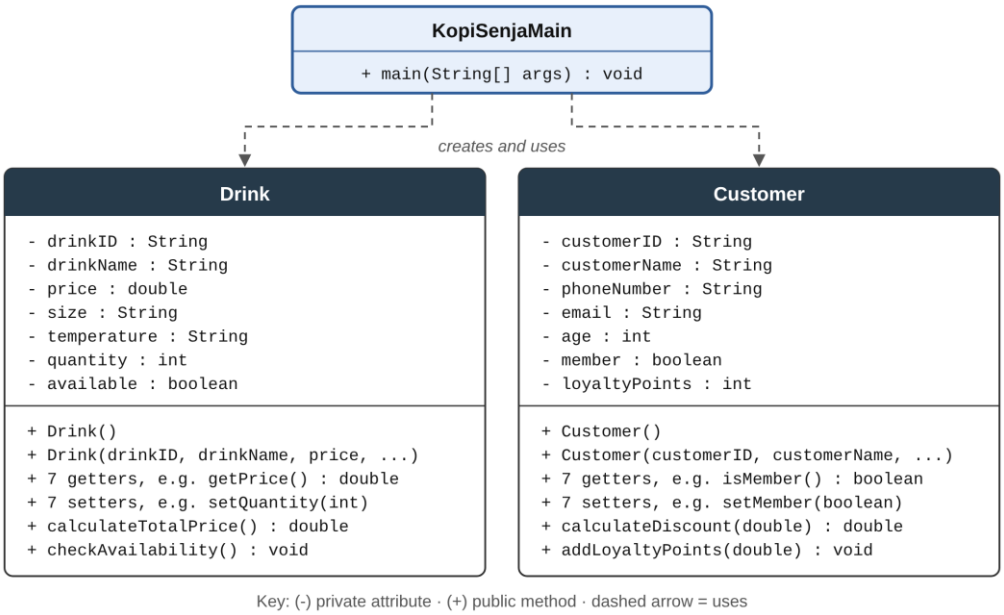


Figure 1. UML class diagram of the Kopi Senja Ordering System

Explanation: The class diagram uses standard UML notation, and each class box is divided into three parts:

1. **Class name** – the top part names the class. A class is a blueprint for creating objects; for example, the Drink class is used to create the Iced Latte, Cappuccino and Americano objects.
2. **Attributes** – the middle part lists the data each object stores, together with its data type: String for text, int for whole numbers, double for decimal values and boolean for true/false values. The minus sign (-) shows that every attribute is private and cannot be accessed directly from outside the class.
3. **Methods (operations)** – the bottom part lists the constructors, getters, setters and additional methods. The plus sign (+) shows that they are public, so other classes such as KopiSenjaMain can call them. Private attributes combined with public methods demonstrate encapsulation.

2.1 Attributes of Each Class

Drink attribute	Type	Customer attribute	Type
`drinkID` – unique drink code	String	`customerID` – unique customer code	String
`drinkName` – name of the drink	String	`customerName` – customer's name	String
`price` – price per cup (RM)	double	`phoneNumber` – contact number	String
`size` – cup size	String	`email` – email address	String
`temperature` – Hot or Iced	String	`age` – age in years	int
`quantity` – cups ordered	int	`member` – membership status	boolean
`available` – can be sold	boolean	`loyaltyPoints` – points collected	int

3.0 OOP PRINCIPLES APPLIED

The table below explains the main OOP terms used in this assignment and shows where each one appears in our system.

OOP term	Meaning	Example in Kopi Senja
Class	A blueprint that defines the attributes and methods shared by all objects of one type.	`Drink`, `Customer`
Object	A specific instance of a class, created with the `new` keyword. Each object holds its own values.	`drink1` (Iced Latte), `drink3` (Americano), `customer`
Attribute	A variable inside a class that stores one property of an object.	`price`, `quantity`, `member`
Method	A block of code inside a class that defines what an object can do.	`calculateTotalPrice()`, `calculateDiscount()`
Constructor	A special method with the same name as the class that runs when an object is created and sets its starting values.	`Drink()` and `Drink(String drinkID, ...)`
Encapsulation	Hiding an object's data by making attributes private and allowing access only through public methods.	All 14 attributes are `private`
Getter / Setter	Public methods that read (get) or change (set) the value of a private attribute.	`getPrice()`, `setQuantity()`

Class vs object. A class is like a recipe card, while an object is an actual cup of coffee made from that recipe. Our program has one Drink class but three Drink objects. Each object has the same attributes but different values; for example, `drink1` costs RM12.00 while `drink3` costs RM8.00.

Object interaction. Objects work together by calling each other's public methods. The main class asks each Drink object for its subtotal using `calculateTotalPrice()`, then passes the total to the Customer object's `calculateDiscount()` method. Neither class needs to know how the other stores its data.

4.0 JAVA SOURCE CODE IMPLEMENTATION

The following excerpts show how the classes were implemented. The complete, fully commented source files (`Drink.java`, `Customer.java` and `KopiSenjaMain.java`) are included in the project ZIP file submitted with this report.

4.1 Constructors

```
// Default constructor initializes a drink with default values.
public Drink() {
    this("D000", "Unknown", 0.0, "Medium", "Iced", 1, false);
}

// Parameterized constructor uses the values provided.
public Drink(String drinkID, String drinkName, double price, String size,
             String temperature, int quantity, boolean available) {
    this.drinkID = drinkID;
    this.drinkName = drinkName;
    this.price = price;
    // ... the remaining four attributes are assigned the same way
}
```

Explanation: The default constructor takes no arguments and gives the object safe starting values. Instead of repeating seven assignments, it calls the parameterized constructor using `this(...)`. The parameterized constructor receives all seven values when an object is created, such as `new Drink("D001", "Iced Latte", 12.00, "Medium", "Iced", 1, true)`. The keyword `this` refers to the current object and separates the attribute (`this.price`) from the parameter with the same name (`price`). The Customer class follows the same pattern, and its default constructor creates a "Guest" customer with the ID "C000", which the main class later updates with setters.

4.2 Encapsulation: Getters and Setters

```
public double getPrice() {
    return price;                    // Returns the drink price.
}

public void setQuantity(int quantity) {
    // Checks whether the quantity is zero or positive.
    if (quantity >= 0) {
        this.quantity = quantity;    // Accepts a valid quantity.
    } else {
        this.quantity = 0;           // Prevents a negative quantity.
    }
}
```

Explanation: `getPrice()` returns the value of the private `price` attribute, giving other classes read-only access. `setQuantity()` updates the number of cups ordered, but first checks that the value is zero or more; a negative number is replaced with 0. The same rule protects `setPrice()`, `setAge()` and `setLoyaltyPoints()`. Each class has seven getters and seven setters, one pair for every attribute.

Why encapsulation matters. If `price` were public, any part of the program could write `drink1.price = -50;` and every bill would be wrong. Because the attribute is private, this line would not even compile, and the only way to change the price is through `setPrice()`. This protects the data, makes errors easier to find, and allows a class to change how it works internally without breaking the code that uses it.

4.3 Additional Methods

```
// Drink: calculates the subtotal for this drink.
public double calculateTotalPrice() {
    return price * quantity;
}

// Drink: displays a message based on availability status.
public void checkAvailability() {
    if (available) {
        System.out.println(drinkName + " is available.");
    } else {
        System.out.println(drinkName + " is unavailable.");
    }
}
```

Explanation: `calculateTotalPrice()` multiplies the price of one drink by the quantity ordered; for example, two Iced Lattes at RM12.00 each give a subtotal of RM24.00. `checkAvailability()` uses an if/else to display whether a drink can be sold. The main program calls it when the customer selects the sold-out Americano.

```
// Customer: members receive a 10% discount.
public double calculateDiscount(double totalPrice) {
    if (member) {
        return totalPrice * 0.10;
    } else {
        return 0;                    // Non-members receive no discount.
    }
}

// Customer: one point for every full RM10 spent.
public void addLoyaltyPoints(double amountSpent) {
    if (member && amountSpent > 0) {
        int pointsEarned = (int) (amountSpent / 10);
        loyaltyPoints += pointsEarned;
    }
}
```

Explanation: `calculateDiscount()` returns 10% of the total for members and zero for non-members, so a RM34.00 order gives a discount of RM3.40. `addLoyaltyPoints()` gives members one point for every full RM10 spent after the discount. The `(int)` cast removes the decimal part, so a payment of RM30.60 earns 3 points rather than 3.06.

5.0 MAIN CLASS AND CONTROL STRUCTURES

KopiSenjaMain contains the `main()` method, where program execution begins. It uses the Scanner class to read keyboard input, while the Drink and Customer objects store the information needed to process the order.

5.1 Object Creation and User Input

```
Scanner input = new Scanner(System.in);
Drink drink1 = new Drink("D001", "Iced Latte", 12.00,
                         "Medium", "Iced", 1, true);
Drink drink3 = new Drink("D003", "Americano", 8.00,
                         "Medium", "Iced", 1, false);
Customer customer = new Customer();         // default constructor

System.out.print("Enter your name: ");
customer.setCustomerName(input.nextLine());  // setter

if (membership.equalsIgnoreCase("yes")) {    // if/else
    customer.setMember(true);
} else {
    customer.setMember(false);
}
```

Explanation: Three Drink objects are created with the parameterized constructor, and the Americano is created with `available` set to `false` to represent a sold-out drink. The Customer object is created with the default constructor, and its name and membership status are then updated through setters instead of accessing the private attributes directly. The if/else statement sets the membership status based on the customer's answer.

5.2 Looping Structure: do-while and Input Checking

```
int choice;
do {
    // Display the menu using getter methods.
    System.out.printf("1. %-12s RM%.2f%n",
                      drink1.getDrinkName(), drink1.getPrice());
    ...
    if (!input.hasNextInt()) {        // Input is not a number.
        System.out.println("Please enter a number!");
        input.next();                 // Discards the invalid input.
        choice = 0;
        continue;                     // Shows the menu again.
    }
    choice = input.nextInt();
    ...
} while (choice != 4);                // Repeats until checkout.
```

Explanation: The do-while loop displays the menu and lets the customer order as many drinks as they want, repeating until option 4 (Checkout) is chosen. A do-while loop suits this task because the menu must be shown at least once. The menu prices are read from the objects using `getDrinkName()` and `getPrice()`, so they always match the stored data. The `hasNextInt()` check rejects text input such as "abc" and shows the menu again instead of crashing the program.

5.3 Selection Structures: switch and Nested if/else

```
switch (choice) {            // Selects a drink based on the menu choice.
    case 1: selectedDrink = drink1; break;
    case 2: selectedDrink = drink2; break;
    case 3: selectedDrink = drink3; break;
    case 4: System.out.println("Proceeding to checkout..."); break;
    default: System.out.println("Invalid choice!");
}

if (selectedDrink != null) {
    if (selectedDrink.isAvailable()) {
        int quantity = input.nextInt();
        if (quantity > 0) {
            selectedDrink.setQuantity(quantity);            // setter
            total += selectedDrink.calculateTotalPrice();   // method
        } else {
            System.out.println("Quantity must be greater than 0.");
        }
    } else {
        selectedDrink.checkAvailability();   // "Americano is unavailable."
    }
}
```

Explanation: The switch statement compares `choice` with each case. Cases 1 to 3 store the matching Drink object in `selectedDrink`, case 4 prints a checkout message, and `default` handles numbers that are not on the menu. The `break` statement stops execution from continuing into the next case. The nested if/else statements then check that a drink was selected, that it is available, and that the quantity is greater than zero. A valid quantity is saved through `setQuantity()`, and the subtotal from `calculateTotalPrice()` is added to the running total. At checkout, the program calls `calculateDiscount()` and `addLoyaltyPoints()` and prints a receipt using getters such as `getCustomerName()` and `getLoyaltyPoints()`.

5.4 Summary of Control Structures and Object Interaction

Structure / method type	Where it is used	Purpose
`if/else`	Membership input, receipt	Set and display member status
`if` with `hasNextInt()`	Menu input	Reject non-numeric input without crashing
`switch`	Menu choice	Select the drink object, checkout or show an error
Nested `if/else`	Order processing	Check selection, availability and quantity
`do-while`	Ordering loop	Repeat the menu until the customer checks out
Setters called	Main class	`setCustomerName()`, `setMember()`, `setQuantity()`
Getters called	Main class	`getDrinkName()`, `getPrice()`, `isAvailable()`, `getCustomerName()`, `isMember()`, `getLoyaltyPoints()`

Program flow: enter name and membership → choose a drink → check availability → enter quantity → calculate subtotal → repeat menu or checkout → calculate discount and loyalty points → display receipt.

6.0 SAMPLE OUTPUT AND TESTING

The program is compiled with `javac Drink.java Customer.java KopiSenjaMain.java` and run with `java KopiSenjaMain`. The output below is from a test run in which a member named Sofia orders two Iced Lattes and one Cappuccino. The repeated menu is omitted to save space.

```
==============================
       WELCOME TO
         KOPI SENJA
==============================
Enter your name: SAFANA
Are you a member? (yes/no): yes

========== MENU ==========
1. Iced Latte    RM12.00
2. Cappuccino    RM10.00
3. Americano     RM8.00
4. Checkout
==========================
Choose your drink: 2
Enter quantity: 1
Cappuccino added to order.
Subtotal: RM10.00

========== MENU ==========
1. Iced Latte    RM12.00
2. Cappuccino    RM10.00
3. Americano     RM8.00
4. Checkout
==========================
Choose your drink: 3
Americano is unavailable.

========== MENU ==========
```

```
========== MENU ==========
1. Iced Latte    RM12.00
2. Cappuccino    RM10.00
3. Americano     RM8.00
4. Checkout
==========================
Choose your drink: 1
Enter quantity: 2
Iced Latte added to order.
Subtotal: RM24.00

========== MENU ==========
1. Iced Latte    RM12.00
2. Cappuccino    RM10.00
3. Americano     RM8.00
4. Checkout
==========================
Choose your drink: 5
Invalid choice!

========== MENU ==========
1. Iced Latte    RM12.00
2. Cappuccino    RM10.00
3. Americano     RM8.00
4. Checkout
==========================
Choose your drink: 4
```

```
Choose your drink: 4
Proceeding to checkout...

==============================
         KOPI SENJA
           RECEIPT
==============================
Customer: SAFANA
Member: Yes
Total: RM34.00
Discount: RM3.40
Final Total: RM30.60
Loyalty Points: 3
==============================
Thank you for visiting Kopi Senja!
```

Explanation: The program rejected the text input "abc" and the sold-out Americano without crashing, added the two valid orders, and applied the 10% member discount. The calculation is: Iced Latte RM12.00 × 2 = RM24.00, Cappuccino RM10.00 × 1 = RM10.00, total RM34.00, discount RM34.00 × 10% = RM3.40, final payment RM30.60, and loyalty points = 30.60 ÷ 10 = 3.

6.1 Test Cases and Results

No.	Test case	Input	Expected result	Actual result	Status
1	Member order	Member; Iced Latte × 2, Cappuccino × 1	Total RM34.00, discount RM3.40, final RM30.60, 3 points	As expected	Pass
2	Non-member order	Non-member; Cappuccino × 1	Final RM10.00, discount RM0.00, 0 points	As expected	Pass
3	Sold-out drink	Choice 3	"Americano is unavailable." and nothing added	As expected	Pass
4	Zero quantity	Choice 1, quantity 0	"Quantity must be greater than 0." and total unchanged	As expected	Pass
5	Invalid menu number	Choice 9	"Invalid choice!" and menu shown again	As expected	Pass
6	Text instead of number	"abc"	"Please enter a number!" and no crash	As expected	Pass

7.0 EVALUATION AND CONCLUSION

7.1 Strengths and Limitations

The main strength of Kopi Senja is its clear separation of responsibilities. Drink stores drink information and performs drink-related calculations, Customer manages customer details and membership benefits, and the main class coordinates the ordering process without accessing any private attribute directly. The program also demonstrates both types of constructors, complete getters and setters, if/else, switch and do-while, and all six test cases produced the expected results.

However, the program has some limitations. The quantity input does not check for non-numeric text, the parameterized constructors assign values directly and therefore skip the validation in the setters, and the menu is fixed in the code. In a future version, the setters could be called from the constructors, the `hasNextInt()` check could be applied to the quantity input, and the drinks could be stored in an array so the menu can be printed with a for loop. An Order class could also be added to print an itemised receipt.

7.2 Conclusion

In conclusion, the Kopi Senja Ordering System shows how object-oriented programming can be used to solve an everyday problem in a simple and organised way. The program is built around two classes, Drink and Customer, each with seven private attributes, two constructors, a full set of getters and setters, and additional methods that handle pricing, availability, discounts and loyalty points. By keeping all attributes private and validating values in the setters, the program applies encapsulation to protect its data from invalid changes.

The KopiSenjaMain class brings these objects together, using if/else statements, a switch statement and a do-while loop to let the customer place an order and receive a receipt, while handling mistakes such as invalid choices, zero quantities and sold-out drinks. Through this assignment, we learned that splitting a program into classes makes the code easier to read, test and improve, because each class is responsible for its own data. Overall, the project gave us practical experience of how the main OOP concepts work together to build a working system.

GitHub Repository

The complete source code for the Kopi Senja Java Coffee Ordering System is available on GitHub. The repository contains Drink.java, Customer.java, KopiSenjaMain.java, and a README file explaining the project and execution instructions.

https://github.com/safanamariyam02-wq/KopiSenja